

CS6310 – Software Architecture & Design

Assignment #3 [100 points]: Pokémon Tournament Project – Phase 3 (v0)

"Welcome to the Poke-Dome!"

Spring 2025 Semester – created by the CS6310 Instructional Team

Functional & Non-Functional Additions

The clients are excited about your work as you enter the final phase of this Pokémon Tournament Manager Project. Given the time you have spent studying the problem space, the clients are now asking you to **make three (3) different additions to the system**, where each addition represents a new functional or non-functional modification:

- Functional modifications affect the way that the domain objects (e.g., Pokémon, battles, tournaments, etc.) operate, behave, and interact; and,
- Non-functional modifications improve the system's overall structure based on one or more architectural perspectives.

The clients will designate one functional or one non-functional modification that you must implement, and then you must select one more functional modification, and one more non-functional modification.

You will work with your teammates during this phase to integrate your earlier individual efforts into a single system and plan how you will implement the designated modifications for your team. You should also give serious thought to the new commands and instructions that you'll need to provide in your interface to support these new capabilities.

The possible functional and non-functional (i.e., architectural areas of consideration) modifications are listed below, and we (i.e., the client) will select from across both types of modifications. ***You can view your assigned modifications through the A3_Assigned_Modifications spreadsheet in PostEm on Canvas.*** The clients are excited about your design proposals as we enter the final phase of the Pokémon project.

Disclaimer

This scenario has been developed solely for this course. Any similarities or differences between this scenario and any of the programs at Georgia Tech programs are purely coincidental.

Project Modifications

Submit your proposed project modifications to your assigned TA via their requested means of communication. Your proposal should contain one approx. six to twelve (~6-12) sentence description for each modification covering the "what you are trying to do" (ADR) section of the modification.

- For the functional & non-functional modification(s), you must use an Architectural Decision Record (ADR) format. ADRs are discussed in more depth in Chapter 19 of Fundamentals of Software Architecture (Ford & Richards, O'Reilly 2020). Also, here are some links that contain more examples of (and details about) ADRs and the process:
 - <https://adr.github.io/>
 - <https://www.ozimmer.ch/practices/2020/05/22/ADDefinitionOfDone.html>
 - <https://medium.com/olzzio/y-statements-10eb07b5a177>
- This isn't intended to be an extensive writing exercise, but rather an opportunity to ensure that you've considered the potential architectural, design and implementation challenges of the assigned modifications.
- "Significant" in this case means that there must be some clear and substantial level of effort that is needed to modify the current design and codebase in order to implement your proposed modifications. The primary things that we will look for when evaluating your proposals are:
 1. Clear identification of the modifications that you will be making. The best way to do this is to use a basic reference architecture for your comparison, such as a relatively simple solution to the previously implemented designs.
 2. Presentation of a relatively strong case of how your modifications will improve the system. For performance-based changes, you can show how the improved system will outperform the basic system - either in theory, or with real data and testing, or both. For other situations, you can present a sequence of actions that highlight how the basic system fails to provide the needed properties, while the improved system overcomes these issues.
 3. Finally, presenting a convincing case for your proposals doesn't mean you must avoid any trade-offs. It's perfectly acceptable to highlight the strengths of your proposals while also discussing some of the difficulties, to include how improving some of the system properties can also have an adverse impact on others.
- You should also review Heilmeier's Catechism below for some issues and considerations that you can address to strengthen your proposal descriptions. These questions are credited to George Heilmeier and are relevant when evaluating research projects or product development efforts. You can find more details about Heilmeier's Catechism at the following link:
https://en.wikipedia.org/wiki/George_H._Heilmeier
 - **What are you trying to do?** Articulate your objectives using absolutely no jargon.
Describe the proposed improvement in a clear and concise manner.
 - **How is it done today, and what are the limits of current practice?**

- **What is new in your approach and why do you think it will be successful?**
In the context of this project, does the current system provide similar capabilities already? Is your proposal a new capability, or an improvement over an existing but limited capability?
- **Who cares?**
- **If you are successful, what difference will it make?**
In this project's context, which simulation users will find this improvement useful, and why/how?
- **What are the risks and the payoffs?**
- **How much will it cost?**
- **How long will it take?**
You must estimate the difficulty involved in implementing this proposed improvement in development resources.
- **What are the midterm and final "exams" to check for success?**
You must provide the "acceptance criteria" for this improvement in terms of clear and quantifiable metrics that can be used to verify that it has been implemented correctly.

You do not have to specifically address every question above for each proposal, but when used appropriately they can help you make proposals as clear, concise, and persuasive as possible.

“Prepare For Trouble And Make It Double!” and Other Hazards (Functional Modifications)

This list is just a subset with basic ideas. If your group has a different modification that would be a good fit for the project, please discuss it with your assigned TA.

[1] Coordinated Team Battles: Battles involving more than 1 vs 1 Pokémon as more of a coordinated team effort.

- Ex: 3 vs 3. This brings a new challenge about how or when to swap Pokémon out before they faint.
- Your Grading TA should be able to setup both teams & fully control 1 of the Pokémon teams, while the other team could still run in a “Random or Seeded Random” fashion with set “Event Thresholds” that could trigger actions to swap, attack, defend, etc... If a seed is set for the Random, your TA will need the ability to customize it as with current battles, & tournaments.

[2] Create Persistent Pokémon: Hidden Pokémon were 1 way of adding Pokémon to your system via a means like DLC (Downloadable Content), but what about being able to create them through your GUI & have them persist in the application?

[3] Add Skill Points & Modifiers: Damage modifiers & elemental strengths & weaknesses, rarity or shiny, adding SP or Skill Points for the cost of using a skill. So, you may not be able to always use that max damage attack if you run out of SP. This also comes with being able to add new skills or types of skills. Ex: Adding a burn mechanic that could do DoT (Damage over Time), or conceptually allowing a Pokémon to attack twice in the same turn.

[4] Support Evolving Pokémon: Leveling up, enhancing stats, tracking the wins / losses for experience. Since Pokémon have a relatively short lifecycle, this would mean figuring out a separate way of persisting their information than an instance that is created / disposed of for every battle. This may come with being able to evolve the Pokémon if it is an option or learn new skills as they level up. Which may come with being able to select which are the Pokémon's current skills from the list of all the skills they have learned.

- Ex: if the Pokémon started with 4 attack skills, and learned 4 more for a total of 8, being able to customize which 4 of those 8 it used in battle.

Architectural Areas of Consideration (Non-Functional Modifications)

We have identified nine (9) non-functional architectural concepts below. These terms and their descriptions were taken from the textbook "Fundamentals of Software Architecture" by Neal Ford & Mark Richards (O'Reilly Media, 2020). **Your task is to determine how you would modify the architecture and design of your system to better address these topics.**

[1] Scalability: Ability for the system to perform and operate as the number of users or requests increases.

As the popularity of the system you have developed grows, it might be deployed across a wider – and possibly worldwide – audience. Consider also that not only might the number of users increase, but the users might be distributed across a larger geographical area. How would you modify the system to better support an exceptionally large number of users being able to access the system concurrently without corrupting the operation of the system?

[2] Performance: Includes stress testing, peak analysis, analysis of the frequency of functions used, capacity required, and response times. Performance acceptance sometimes requires an exercise of its own, taking months to complete.

If there are hundreds of thousands, or millions, of streaming service viewers using the system on a "round the clock/24 hour" basis, then the data collected in the system will generally increase. The increase in the size of the data set and the number of transactions will potentially increase the amount of time needed to display reports, or to perform checks to ensure that transactions can be executed correctly. How would you modify the system to maintain and/or improve the performance of the system?

[3] Robustness: Ability to handle error and boundary conditions while running if the internet connection goes down or if there is a power outage or hardware failure.

There have not been many error handling requirements so far. This is an opportunity to improve the error handling of the system and much more. Consider the possibility of the program crashing – for example, power being lost – while executing a program run. How would you modify the system to allow your system to resume execution after such an event without major delays, data corruption, etc.?

[4] Configurability: Ability for the end users to easily change aspects of the software's configuration (through usable interfaces).

We implemented a relatively simple Command Line Interface in earlier assignments, so this is an opportunity not only to generate a better, richer interface, but also to address things like the ability to set the starting month and date of the program run instead of hardcoding it into the program. Similarly, the implementation of some of your other architectural and design changes might create other configuration parameters that need to be changed. How would you modify the system to allow users to manage these configuration parameters as easily as possible?

[5] Archivability: Will the data need to be archived or deleted after a period of time? (For example, customer accounts are to be deleted after three months or marked as obsolete and archived to a secondary database for future access.)

Suppose the data being collected and analyzed must be stored for a certain time period. This might be driven by a legal requirement, a policy requirement, or a technical limitation, but either way the system must be implemented to manage the storage of this information. And, in some cases, there must also be ways to ensure that the information is purged after a certain amount of time.

These next two architectural concepts are "intertwined" in that it's difficult to consider one of them in isolation, because implementing one of them effectively all but requires that you consider the impacts (and dependencies) on the other. I'm listing these two terms here as a "bundle".

[6] Authentication/Authorization

- **Authentication:** Security requirements to ensure users are who they say they are.
- **Authorization:** Security requirements to ensure users can access only certain functions within the application (by use case, subsystem, webpage, business rule, field level, etc.).

Authentication and authorization are often essential if you are going to enforce any kind of security and/or privacy protocols.

[7] Privacy: Ability to hide transactions from internal company employees (encrypted transactions so even DBAs and network architects cannot see them).

[8] Security: Does the data need to be encrypted in the database? Encrypted for network communication between internal systems? What type of authentication needs to be in place for remote user access?

Also, the public has become much more aware of the need for data privacy, and there are various things that can be done to ensure that user's data – especially medically and/or financially sensitive data – is accessed only by authorized users for its intended purposes.

[9] *Auditability: Ability to track the "activity within the system" (e.g., which commands were executed by which users) in such a way that a user's activity records can only be reviewed by

personnel in appropriate roles, such as a Security Administrator; and, that a user cannot remove or otherwise corrupt their own activity records.

**I included this one from other sources including my own personal experience.*

The unifying theme of all these architectural considerations is making the system more "trustworthy" against the case of deliberate, adversarial threats as well as accidental, non-adversarial issues that might corrupt the data and/or operations of the system. How would you modify the system to improve its overall level of trustworthiness?

Closing Comments & Suggestions

This is the information provided by the customer so far. We (the OMSCS 6310 Team) will conduct Office Hours where you will be permitted to ask us questions to clarify the client's intent, etc. We will answer some of the questions, but we will not necessarily answer all of them. Also, though this current version of the system is the "core" of the system going forward, our clients will add, update, and remove some of the requirements over the span of the course. One of your main tasks will be to ensure that your architectural documents and related artifacts remain consistent with the problem requirements – and with your system implementations – over time.

Quick Reminder on Collaborating with Others

Please use Ed Discussion for your questions and/or comments, and post publicly whenever it is appropriate. If your questions or comments contain information that specifically provides an answer for some part of the assignment, then please make your post private first, and we (the OMSCS 6310 Team) will review it and decide if it is suitable to be shared with the larger class. Best of luck to you with this assignment, and please contact us if you have questions or concerns.